ΕΛΛΗΝΙΚΗ ΔΗΜΟΚΡΑΤΙΑ
Εθνικόν και Καποδιστριακόν Πανεπιστήμιον Αθηνών
ΙΔΡΥΘΕΝ ΤΟ 1837

Εισαγωγή στον Προγραμματισμό

Εργασία #1

Νοέμβριος 2026

Στόχος αυτής της εργασίας είναι να γράψουμε ένα μεγαλύτερο πρόγραμμα σε γλώσσα C που επεξεργάζεται εικόνες. Συγκεκριμένα, στοχεύουμε σε εμπειρία με τα ακόλουθα:

1. Χρήση ορισμάτων από την γραμμή εντολών
2. Χρήση πινάκων και συμβολοσειρών
3. Χρήση stdin / stdout / stderr
4. Ανάγνωση, μετατροπή και γραφή δυαδικών (και μη) δεδομένων
5. Αποδοτική διαχείριση δεδομένων σε χρόνο και μνήμη

Υποβολή Εργασίας. Όλες οι υποβολές των εργασιών θα γίνουν μέσω GitHub και συγκεκριμένα στο github.com/progintro [17]. Προκειμένου να ξεκινήσεις, μπορείς να δεχτείς την άσκηση με αυτήν την: πρόσκληση [18].

Σημαντικό: Στην εργασία, όπως και σε όλες τις εργασίες του μαθήματος, δεν επιτρέπεται η χρήση τεχνητής νοημοσύνης για να ολοκληρώσετε τις ασκήσεις. Οποιαδήποτε χρήση θα οδηγήσει σε μηδενισμό στο μάθημα.

1. Φίλτρα για Όλους/ες (Photoula - 300 Μονάδες)

Ίσως σου έχει τύχει να χρησιμοποιήσεις φίλτρα σε φωτογραφίες; Η διαδικασία είναι σχετικά απλή, βγάζεις μια φωτογραφία με το κινητό σου, πατάς ένα κουμπί και η φωτογραφία σου μετατρέπεται στην στιγμή: ασπρόμαυρη, ρετρό, με αυξημένο contrast ή ό,τι άλλο θες (σήμερα υπάρχουν φίλτρα για τα πάντα)! Πώς λειτουργεί όμως ένα φίλτρο; Και τι συμβαίνει στο κινητό μας όταν πατήσουμε ένα από αυτά τα φίλτρα; Αυτό θα είναι και το αντικείμενο αυτής της άσκησης όπου θα υλοποιήσουμε ένα νέο, αποδοτικό αλλά και παραμετροποιήσιμο φίλτρο για φωτογραφίες!

Για να φτιάξουμε ένα τέτοιο φίλτρο χρειαζόμαστε πρόσβαση στα ίδια τα εικονοστοιχεία (χρώματα/pixel) της φωτογραφίας. Δεν ξέρω αν έχετε προσπαθήσει να ανοίξετε μια εικόνα τύπου jpg ή png με κειμενογράφο. Αν όχι, δοκιμάστε το: αντί για εικονοστοιχεία θα δείτε ατελείωτες γραμμές από ακαταλαβίστικα σύμβολα, αφού τα αρχεία αυτά είναι δυαδικά και συμπιεσμένα με περίπλοκους αλγορίθμους. Για αυτό θα δουλέψουμε με ένα format που μας δίνει απευθείας τα δεδομένα της εικόνας και που μπορεί να γραφτεί ακόμα και ως κείμενο! Η εφεύρεση αυτή, ήρθε στα μέσα της δεκαετίας του 1980 όταν δεν υπήρχε αξιόπιστος τρόπος να στείλει κανείς δυαδικά αρχεία μέσω email: ό,τι δεν ήταν κείμενο ASCII κινδύνευε να φτάσει αλλοιωμένο. Ο Jef Poskanzer [4] σκέφτηκε το εξής: αφού το email μεταφέρει με ασφάλεια μόνο κείμενο, ας γράψουμε και τις εικόνες σαν κείμενο! Από αυτήν την ιδέα γεννήθηκε το format PBM (Portable Bit Map) για εικόνες και, στην συνέχεια, ολόκληρη η οικογένεια του *Netpbm format* [7, 8] με την οποία θα ασχοληθούμε. Πριν προχωρήσουμε στις τεχνικές προδιαγραφές της εργασίας, παραθέτουμε μια περιγραφή της μορφοποίησης αυτών των αρχείων.

Στον υπολογιστή, μια εικόνα αποτελείται από μια ορθογώνια διάταξη εικονοστοιχείων (pixels) [11], σε M γραμμές και N στήλες (οπότε το μέγεθος της εικόνας είναι $N \times M$, με το N να είναι το πλάτος της και το M το ύψος της), όπου για κάθε εικονοστοιχείο έχουμε αποθηκευμένη την πληροφορία που προσδιορίζει πώς θα πρέπει να εμφανιστεί. Το Netpbm format υποστηρίζει τρία είδη εικόνων:

- **Έγχρωμες εικόνες** (μορφή ppm, από το Portable Pixel Map), όπου κάθε εικονοστοιχείο κωδικοποιείται από τρεις ακεραίους που εκφράζουν τις ποσότητες των βασικών χρωμάτων κόκκινο (R), πράσινο (G) και μπλε (B)—εξ ου και τα αρχικά RGB [14] που συναντάμε παντού όταν ασχολούμαστε με χρώματα στον υπολογιστή. Συνήθως, αλλά όχι πάντοτε, οι αριθμοί αυτοί είναι μη προσημασμένοι ακέραιοι του ενός byte, δηλαδή κυμαίνονται από 0 έως 255.

- **Εικόνες με αποχρώσεις του γκρι** (μορφή pgm, από το Portable Gray Map) [1], όπου κάθε εικονοστοιχείο κωδικοποιείται από έναν μη προσημασμένο ακέραιο, συνήθως από 0 έως 255. Η τιμή 0 αντιστοιχεί στο απόλυτο μαύρο, η μέγιστη τιμή (συνήθως 255) στο απόλυτο άσπρο και οι ενδιάμεσες τιμές σε αποχρώσεις του γκρι.

- **Απολύτως ασπρόμαυρες εικόνες** (μορφή pbm, από το Portable Bit Map), όπου κάθε εικονοστοιχείο κωδικοποιείται με ένα μόνο bit: το 0 είναι άσπρο και το 1 μαύρο. Ναι, καλά διαβάσατε—εδώ το 0 είναι το *αντίθετο* από ό,τι στις εικόνες pgm!

Κάθε ένα από τα τρία είδη μπορεί να αποθηκευτεί με δύο τρόπους. Στην *δυαδική* κωδικοποίηση, για ένα έγχρωμο εικονοστοιχείο χρειάζονται ακριβώς τρία bytes (ένα για κάθε χρώμα, υπό την προϋπόθεση ότι η μέγιστη τιμή δεν ξεπερνά το 255), για ένα εικονοστοιχείο σε απόχρωση του γκρι χρειάζεται ένα byte (πάλι με μέγιστη τιμή το 255), ενώ στις απολύτως ασπρόμαυρες εικόνες χωράνε οκτώ εικονοστοιχεία σε

ένα byte, αφού το καθένα χρειάζεται μόνο ένα bit. Στην κωδικοποίηση *σε μορφή κειμένου*, κάθε ακέραια τιμή γράφεται με τους ASCII κωδικούς των ψηφίων της στο δεκαδικό σύστημα. Για παράδειγμα, ο αριθμός 230 χρειάζεται ένα byte στην δυαδική αναπαράσταση αλλά τρία bytes σε μορφή κειμένου, τους ASCII κωδικούς των ψηφίων 2, 3 και 0. Προφανώς, η δυαδική αναπαράσταση είναι πιο οικονομική, αλλά η αναπαράσταση σε μορφή κειμένου είναι αναγνώσιμη από τον άνθρωπο.

Ένα αρχείο Netpbm έχει την ακόλουθη δομή, με την σειρά:

- Ο *μαγικός αριθμός*, που χαρακτηρίζει τον τύπο της εικόνας και την κωδικοποίηση, γραμμένος με τους ASCII κωδικούς των δύο χαρακτήρων του:

  P1 (ασπρόμαυρη εικόνα σε μορφή κειμένου)
  P2 (εικόνα με αποχρώσεις του γκρι σε μορφή κειμένου)
  P3 (έγχρωμη εικόνα σε μορφή κειμένου)
  P4 (ασπρόμαυρη εικόνα σε δυαδική κωδικοποίηση)
  P5 (εικόνα με αποχρώσεις του γκρι σε δυαδική κωδικοποίηση)
  P6 (έγχρωμη εικόνα σε δυαδική κωδικοποίηση)

- *Λευκό διάστημα*, δηλαδή ένας ή περισσότεροι λευκοί χαρακτήρες (κενά, στηλογνώμονες, αλλαγές γραμμής και carriage return `'\r'`).

- Το πλάτος της εικόνας, με τους ASCII κωδικούς των ψηφίων του στο δεκαδικό σύστημα, και στην συνέχεια λευκό διάστημα.

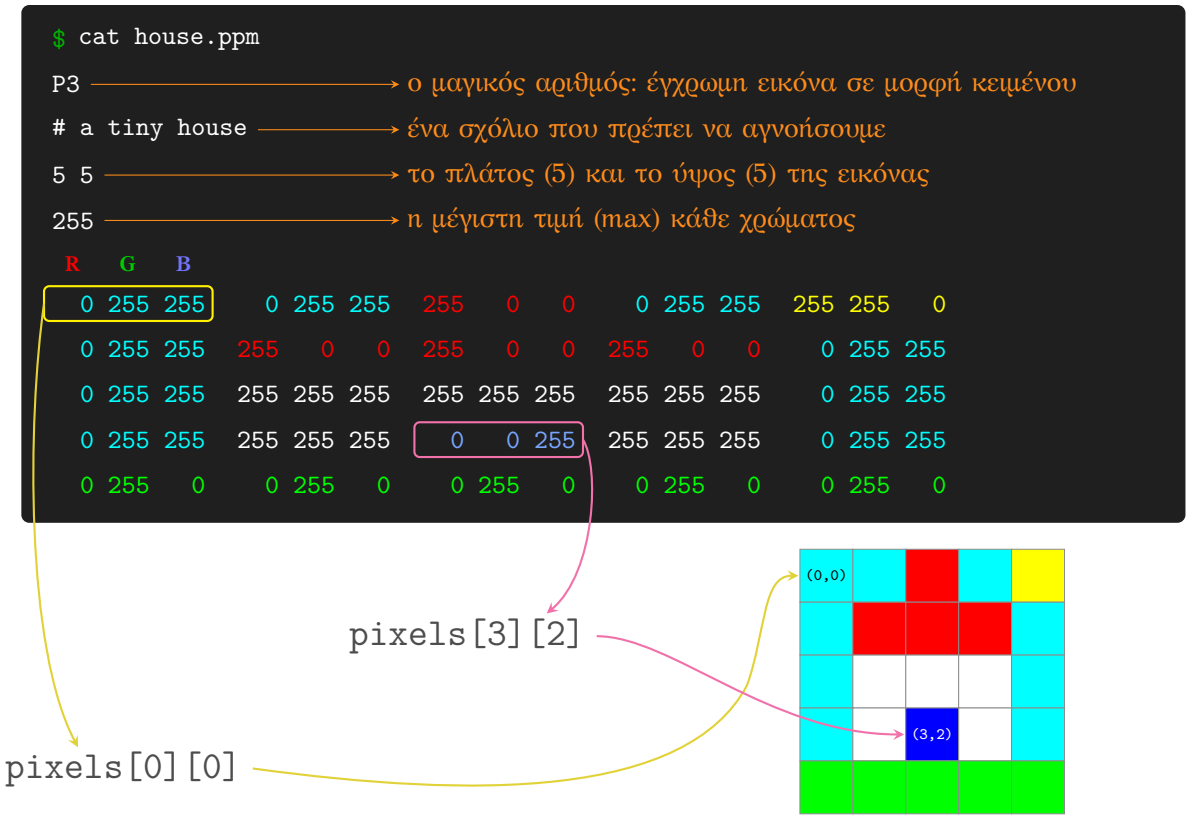
- Το ύψος της εικόνας, με τον ίδιο τρόπο όπως το πλάτος.

- **Μόνο** για έγχρωμες εικόνες και εικόνες με αποχρώσεις του γκρι: λευκό διάστημα και η μέγιστη τιμή *max* που μπορεί να έχει ένας ακέραιος που κωδικοποιεί ένα χρώμα ή μια απόχρωση του γκρι, και αυτή με τους ASCII κωδικούς των ψηφίων της. Η τιμή αυτή είναι πάντα θετική ($max > 0$). Στις απολύτως ασπρόμαυρες εικόνες δεν χρειάζεται, αφού οι τιμές είναι καθορισμένες, 0 ή 1.

- Ένας *μοναδικός* λευκός χαρακτήρας στην περίπτωση δυαδικής κωδικοποίησης ή λευκό διάστημα στην περίπτωση κωδικοποίησης σε μορφή κειμένου.

- Τα δεδομένα της εικόνας, κατά γραμμές από πάνω προς τα κάτω και σε κάθε γραμμή κατά στήλες από αριστερά προς τα δεξιά. Σε μορφή κειμένου, οι ακέραιοι διαχωρίζονται με λευκό διάστημα. Στην δυαδική κωδικοποίηση ασπρόμαυρων εικόνων, όπου οκτώ εικονοστοιχεία χωράνε σε ένα byte, αν το πλάτος της εικόνας δεν είναι πολλαπλάσιο του οκτώ, το τελευταίο byte **κάθε γραμμής** συμπληρώνεται με bits 1 (μαύρο εικονοστοιχείο).

Υπάρχει και μια μικρή παγίδα: μετά το λευκό διάστημα που ακολουθεί τον μαγικό αριθμό και πριν από τις διαστάσεις της εικόνας, μπορεί να υπάρχουν και *σχόλια*. Ένα σχόλιο εκτείνεται από έναν χαρακτήρα `'#'` έως την επόμενη αλλαγή γραμμής



Σχήμα 1: Ένα ολόκληρο αρχείο ppm σε μορφή κειμένου, χρωματισμένο εδώ για να φαίνεται η αντιστοιχία. Μετά την κεφαλίδα, κάθε τριάδα αριθμών είναι ένα εικονοστοιχείο με τις τιμές R, G και B και οι τριάδες διατάσσονται κατά γραμμές σε ένα δισδιάστατο πλέγμα, που βλέπουμε κάτω δεξιά σε μεγεθυμένη μορφή. Με pixels[γραμμή][στήλη] σημειώνουμε τη θέση ενός εικονοστοιχείου.

('\n'). Μπορεί να υπάρχει σχόλιο στην ίδια γραμμή με τον μαγικό αριθμό, αλλά μπορεί να ακολουθούν και ολόκληρες γραμμές που περιέχουν μόνο σχόλια, μέχρι την πληροφορία πλάτους και ύψους της εικόνας.

Η παραπάνω περιγραφή καλύπτει όσα χρειαζόμαστε για την εργασία και είναι λίγο πιο περιορισμένη από το πλήρες format: για παράδειγμα, όλες οι εικόνες της εργασίας θα έχουν $max \leq 255$. Για οποιαδήποτε περαιτέρω απορία, μπορείτε να ανατρέξετε στις επίσημες προδιαγραφές των μορφών ppm [12], pgm [10] και pbm [9]—άλλωστε, μια εκφώνηση μπορεί πάντα να έχει και λάθη!

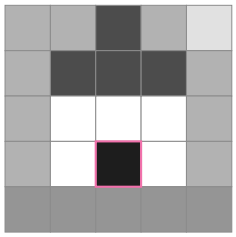
Ας δούμε πώς δένουν όλα αυτά σε ένα παράδειγμα. Το αρχείο `house.ppm` που βλέπουμε στο Σχήμα 1 είναι μια έγχρωμη εικόνα 5×5 σε μορφή κειμένου και πιάνει μόλις 346 bytes. Θυμάστε τι βλέπαμε όταν ανοίγαμε ένα jpg με κειμενογράφο; Εδώ κάθε χρώμα κάθε εικονοστοιχείου είναι ένας αριθμός που μπορούμε να διαβάσουμε και να αλλάξουμε με το χέρι! Ανοίξτε το, αλλάξτε έναν αριθμό και δείτε το αποτέλεσμα!

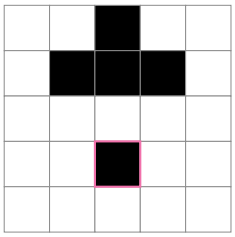
Στο Σχήμα 2 βλέπουμε το ίδιο σπιτάκι και στις άλλες δύο ASCII μορφές. Στο pgm

```
$ cat house.pgm
P2 ──────→ γκρι, σε μορφή κειμένου
5 5 ──────→ πλάτος και ύψος
255 ──────→ η μέγιστη τιμή (max)
178  178   76  178  225
178   76   76   76  178
178  255  255  255  178
178  255   29  255  178
149  149  149  149  149
```

```
$ cat house.pbm
P1 ──────→ ασπρόμαυρη, σε κείμενο
5 5 ──────→ πλάτος και ύψος
           δεν υπάρχει γραμμή max
0 0 1 0 0
0 1 1 1 0
0 0 0 0 0
0 0 1 0 0
0 0 0 0 0

1 = μαύρο
0 = άσπρο
```





Σχήμα 2: Το σπιτάκι του Σχήματος 1 ως εικόνα pgm (αριστερά) και ως εικόνα pbm (δεξιά), πάλι σε μορφή κειμένου. Η πόρτα (γραμμή 3, στήλη 2) έχει τιμή 29 στο pgm, δηλαδή σκούρο γκρι, και τιμή 1 στο pbm, δηλαδή μαύρο.

κάθε εικονοστοιχείο είναι πλέον ένας μόνο αριθμός, ενώ στο pbm έχουν μείνει μόνο το 0 και το 1 και η κεφαλίδα δεν έχει καν μέγιστη τιμή. Το πώς προκύπτουν αυτές οι τιμές θα το δούμε παρακάτω.

Ας δούμε τώρα και τα περιεχόμενα μερικών μεγαλύτερων αρχείων. Μπορούμε να δούμε τα δυαδικά περιεχόμενα οποιουδήποτε αρχείου χρησιμοποιώντας την εντολή `od`. Με τις παρακάτω εντολές βλέπουμε ανά 16άδες τα περιεχόμενα των αρχείων στο δεκαεξαδικό σύστημα, καθώς και τους χαρακτήρες που αντιστοιχούν στους κωδικούς αυτούς (έχουμε παραλείψει τους χαρακτήρες από το σημείο που αρχίζουν τα δεδομένα της εικόνας):

```
$ od -tx1c NIbin.ppm
0000000  50  36  0a  23  20  43  72  65  61  74  65  64  20  62  79  20
          P   6  \n   #       C   r   e   a   t   e   d       b   y
0000020  49  72  66  61  6e  56  69  65  77  0a  32  30  34  38  20  31
          I   r   f   a   n   V   i   e   w  \n   2   0   4   8       1
0000040  35  33  36  0a  32  35  35  0a  4a  85  c9  49  86  c9  48  85
          5   3   6  \n   2   5   5  \n  ..............................
0000060  ................................................................
```

```
$ od -tx1c NIbin.pgm
0000000  50  35  0a  32  30  34  38  20  31  35  33  36  0a  32  35  35
          P   5  \n   2   0   4   8       1   5   3   6  \n   2   5   5
0000020  0a  7b  7b  7a  77  74  6f  6b  6a  78  73  71  75  73  6f  6f
         \n  ................................................................
0000040  ................................................................

$ od -tx1c NIbin.pbm
0000000  50  34  0a  32  30  34  38  20  31  35  33  36  0a  ff  ff  ff
          P   4  \n   2   0   4   8       1   5   3   6  \n  ..........
0000020  ................................................................
```

Σημείωση

Πέρα από την εντολή od, υπάρχουν και άλλες εντολές όπως οι: (1) xxd, (2) hexdump, (3) hexedit που προσφέρουν αντίστοιχες δυνατότητες σε Unix-like συστήματα (Linux, MacOS, WSL κοκ). Για να *συγκρίνετε* δύο δυαδικά αρχεία, μπορείτε να χρησιμοποιήσετε την εντολή cmp.
Για περιβάλλοντα Windows, υπάρχουν ελεύθερα διαθέσιμα προγράμματα που λειτουργούν παρόμοια (π.χ. https://mh-nexus.de/en/hxd/), ενώ για να εμφανίσετε αρχεία Netpbm μπορείτε να χρησιμοποιήσετε το ελεύθερα διαθέσιμο IrfanView [3] ή το GIMP.

Φτάσαμε λοιπόν στο ζητούμενο αυτής της άσκησης: να γράψουμε ένα πρόγραμμα C το οποίο υλοποιεί το δικό μας retro φίλτρο που μετατρέπει έγχρωμες εικόνες σε εικόνες με αποχρώσεις του γκρι και εικόνες με αποχρώσεις του γκρι σε ασπρόμαυρες. Η υλοποίησή σας θα χρειάζεται να χειριστεί αρκετές περιπτώσεις: τρία είδη εικόνων, δύο κωδικοποιήσεις, σχόλια μέσα στις κεφαλίδες και εικονοστοιχεία που μοιράζονται το ίδιο byte. Σας προτείνουμε να αναπτύξετε το πρόγραμμά σας σταδιακά: υλοποιήστε τις μετατροπές μία-μία και, αφού βεβαιωθείτε ότι τα αποτελέσματα είναι σωστά, συνεχίστε με την επόμενη. Οι τεχνικές προδιαγραφές της υλοποίησης ακολουθούν.

Τεχνικές Προδιαγραφές

- Όνομα Προγράμματος: photoula
- Repository Name: progintro/hw1-2026-<YourUsername>
- README Filepath: photoula/README.md
- Αρχείο C (Filepath): photoula/src/photoula.c
- Το αρχείο C που θα υποβληθεί πρέπει να μεταγλωττίζεται χωρίς ειδοποιήσεις για λάθη και με κωδικό επιστροφής (exit code) που να είναι 0. Συγκεκριμένα,

το αρχείο σας **πρέπει** να μπορεί να μεταγλωττιστεί επιτυχώς με την ακόλουθη εντολή σε ένα από τα μηχανήματα του εργαστηρίου (linuxXY.di.uoa.gr):

```
gcc -O3 -Wall -Wextra -Werror -pedantic -o photoula photoula.c
```

- Όλα τα περιεχόμενα των εικόνων θα δίνονται στο πρόγραμμά σας μέσω της πρότυπης εισόδου (stdin) και πρέπει να διαβάζονται με χρήση της συνάρτησης getchar. Δεν θα γίνουν δεκτές υλοποιήσεις όπου χρησιμοποιήθηκε κάποια άλλη συνάρτηση εισόδου εκτός της getchar.
- Στην πρότυπη έξοδο (stdout) πρέπει να τυπώνονται **αυστηρά** δεδομένα εικόνων και τίποτα άλλο (με μόνη εξαίρεση το `--help`). Όλα τα μηνύματα λάθους τυπώνονται στο πρότυπο ρεύμα εξόδου σφάλματος (stderr).
- Το πρόγραμμά σας πρέπει να επιστρέφει κωδικό εξόδου (exit code) 0 όταν ολοκλήρωσε την επεξεργασία του επιτυχώς–διαφορετικά πρέπει να επιστρέφει με κωδικό εξόδου 1.
- Το πρόγραμμά σας πρέπει να διαχειρίζεται τόσο δυαδικές κωδικοποιήσεις όσο και κωδικοποιήσεις σε μορφή κειμένου.
- Μπορείτε να χρησιμοποιήσετε πίνακες, όμως λόγω του περιορισμού μνήμης δεν χωράει ολόκληρη η εικόνα στην μνήμη: οι μετατροπές σας πρέπει να γίνονται καθώς διαβάζετε την είσοδο.
- **Περιορισμοί Μνήμης**. Το πρόγραμμά σας πρέπει να χρησιμοποιεί λιγότερο από 3MB μνήμης ανεξαρτήτως των ορισμάτων ή της εικόνας που δίνεται.
- **Περιορισμοί Χρόνου**. Το πρόγραμμά σας πρέπει να ολοκληρώνει κάθε εκτέλεση του σε λιγότερο από 1 δευτερόλεπτο.

Ορίσματα Γραμμής Εντολών (20% της βαθμολογίας)

Το ποια μετατροπή θα κάνει το πρόγραμμά σας το αποφασίζει ο χρήστης από την γραμμή εντολών. Η σύνταξη του προγράμματος είναι:

```
./photoula --to ppm|pgm|pbm [--threshold T]
           [--encoding ascii|binary] [--r R] [--g G] [--b B]
./photoula --help
```

όπου τα ορίσματα μέσα σε αγκύλες είναι προαιρετικά και συγκεκριμένα:

- `--to type`: Ο τύπος της εικόνας εξόδου (ppm, pgm ή pbm). Το όρισμα είναι υποχρεωτικό.

- `--threshold T`: Το κατώφλι για τις ασπρόμαυρες εικόνες (δείτε παρακάτω). Επιτρέπεται μόνο μαζί με το όρισμα `--to pbm`.
- `--encoding`, `--r`, `--g`, `--b`: Περιγράφονται στο Bonus στο τέλος της εκφώνησης.
- `--help`: Τυπώνει στο stdout μια σύντομη περιγραφή του προγράμματος και τερματίζει με κωδικό εξόδου 0, χωρίς να διαβάσει καθόλου την είσοδο. Η περιγραφή είναι δική σας, αρκεί η πρώτη γραμμή να ξεκινά με `Usage:`. Το `--help` πρέπει να λειτουργεί όπου και αν εμφανιστεί στα ορίσματα.

Τα ορίσματα μπορούν να δοθούν με οποιαδήποτε σειρά. Αν λείπει το `--to`, αν δοθεί άγνωστο όρισμα, αν κάποιο όρισμα δοθεί δύο φορές ή αν η τιμή ενός ορίσματος δεν είναι έγκυρη, το πρόγραμμά σας πρέπει να τυπώνει στο stderr το μήνυμα:

```
Usage: ./photoula --to <ppm|pgm|pbm> [options]
```

και να τερματίζει με κωδικό εξόδου 1. Οι μετατροπές που πρέπει να υποστηρίζονται είναι οι ακόλουθες:

Είσοδος	`--to ppm`	`--to pgm`	`--to pbm`
ppm	αντιγραφή	φωτεινότητα	φωτεινότητα και κατώφλι
pgm	σφάλμα	αντιγραφή	κατώφλι
pbm	σφάλμα	σφάλμα	αντιγραφή

Δεν υποστηρίζεται χρωματισμός εικόνων: για κάθε μετατροπή που θα απαιτούσε εγχρωμάτιση, δηλαδή pgm → ppm, pbm → ppm και pbm → pgm, το πρόγραμμά σας πρέπει να τυπώνει στο stderr `Unsupported conversion` και να τερματίζει με κωδικό εξόδου 1. Αν ο τύπος εξόδου είναι ίδιος με τον τύπο της εισόδου, η εικόνα αντιγράφεται στην έξοδο. Προσοχή όμως: και σε αυτήν την περίπτωση η είσοδος ελέγχεται ολόκληρη, και αν δεν είναι σύμφωνη με τις προδιαγραφές το πρόγραμμά σας τυπώνει το μήνυμα λάθους που περιγράφεται παρακάτω. Επίσης, η έξοδος είναι η ίδια εικόνα αλλά όχι απαραίτητα το ίδιο αρχείο, αφού η κεφαλίδα της εξόδου γράφεται πάντα όπως περιγράφεται αμέσως παρακάτω (για παράδειγμα, χωρίς σχόλια).

Κεφαλίδα. Στην έξοδο, η κεφαλίδα γράφεται χωρίς σχόλια, όπως στα αρχεία NIbin.pgm και NIbin.pbm των παραδειγμάτων με την εντολή od παραπάνω (και όχι όπως στο NIbin.ppm, που έχει σχόλιο μετά τον μαγικό αριθμό): ο μαγικός αριθμός, αλλαγή γραμμής, το πλάτος και το ύψος χωρισμένα με ένα κενό, αλλαγή γραμμής και, όπου υπάρχει, η μέγιστη τιμή με μια τελευταία αλλαγή γραμμής. Η μέγιστη τιμή παραμένει ίδια με αυτήν της εισόδου. Αν δεν ζητηθεί διαφορετικά, η κωδικοποίηση της εξόδου είναι ίδια με αυτήν της εισόδου: δυαδική είσοδος δίνει δυαδική έξοδο και είσοδος σε μορφή κειμένου δίνει έξοδο σε μορφή κειμένου.

Τέλος, για κάθε είσοδο εκτός προδιαγραφών, για οποιονδήποτε λόγο (π.χ. άγνωστος μαγικός αριθμός, λάθος κεφαλίδα, τιμή μεγαλύτερη από το max ή ελλιπή δεδομένα), το πρόγραμμά σας πρέπει να τυπώνει στο stderr `Input error!` και να

τερματίζει με κωδικό εξόδου 1. Ένα λάθος μπορεί να εντοπιστεί και στη μέση της εικόνας, όταν το πρόγραμμά σας έχει ήδη γράψει μέρος της εξόδου. Σε κάθε περίπτωση λάθους, βαθμολογούνται μόνο το μήνυμα στο stderr και ο κωδικός εξόδου, και όχι ό,τι έχει ήδη γραφτεί στο stdout. Παραδείγματα εκτέλεσης ακολουθούν:

```
$ ./photoula
Usage: ./photoula --to <ppm|pgm|pbm> [options]
$ echo $?
1
$ ./photoula --to gif < NIbin.ppm
Usage: ./photoula --to <ppm|pgm|pbm> [options]
$ ./photoula --to pgm --threshold 100 < NIbin.ppm
Usage: ./photoula --to <ppm|pgm|pbm> [options]
$ ./photoula --to ppm < NIbin.pgm
Unsupported conversion
$ echo $?
1
$ echo "trust me" | ./photoula --to pgm
Input error!
$ echo $?
1
```

Από Έγχρωμη Εικόνα σε Αποχρώσεις του Γκρι (40% της βαθμολογίας)

Με το όρισμα `--to pgm`, το πρόγραμμά σας μετατρέπει έγχρωμες εικόνες σε εικόνες με αποχρώσεις του γκρι. Πώς όμως θα διαλέξουμε το γκρι ενός εικονοστοιχείου; Το μάτι μας δεν είναι το ίδιο ευαίσθητο σε όλα τα χρώματα: βλέπει το πράσινο πολύ πιο φωτεινό από το μπλε. Για αυτό, αντί για τον μέσο όρο των τριών τιμών, χρησιμοποιούμε τη μέθοδο της *φωτεινότητας* (luminosity) [5] με συντελεστές που προέρχονται από πρότυπο της ψηφιακής τηλεόρασης [13]. Σε ένα έγχρωμο εικονοστοιχείο με τιμές R, G και B αντιστοιχεί η απόχρωση του γκρι:[1]

$$\left\lfloor \frac{299 \cdot R + 587 \cdot G + 114 \cdot B}{1000} \right\rfloor$$

Προσοχή: ο υπολογισμός πρέπει να δίνει ακριβώς το ίδιο αποτέλεσμα με τον παραπάνω τύπο, οπότε σκεφτείτε καλά αν θέλετε να εμπιστευτείτε αριθμούς κινητής υποδιαστολής για αυτήν την δουλειά. Παράδειγμα εκτέλεσης:

```
$ ./photoula --to pgm < NIbin.ppm > NIbin.pgm
```

[1] Το $\lfloor X \rfloor$ είναι το ακέραιο μέρος του X.

Από Αποχρώσεις του Γκρι σε Ασπρόμαυρη Εικόνα (40% της βαθμολογίας)

Με το όρισμα `--to pbm`, το πρόγραμμά σας κάνει το τελευταίο βήμα: κάθε απόχρωση του γκρι γίνεται είτε απόλυτα άσπρη είτε απόλυτα μαύρη, με την τεχνική του *κατωφλίου* (thresholding) [15]. Αν η τιμή της απόχρωσης του γκρι είναι **μεγαλύτερη** από το κατώφλι T, το εικονοστοιχείο εξόδου είναι άσπρο (τιμή 0), αλλιώς είναι μαύρο (τιμή 1). Αν ο χρήστης δεν δώσει `--threshold`, το κατώφλι είναι:

$$T = \left\lfloor \frac{max + 1}{2} \right\rfloor$$

όπου max είναι η μέγιστη τιμή που μπορεί να έχει ένα εικονοστοιχείο της εισόδου. Το κατώφλι που δίνει ο χρήστης πρέπει να είναι ακέραιος με $0 \leq T \leq max$. Αν δεν είναι ακέραιος ή είναι αρνητικός, πρόκειται για λάθος στα ορίσματα. Το max όμως το μαθαίνουμε μόνο αφού διαβάσουμε την κεφαλίδα, οπότε για $T > max$ το πρόγραμμά σας τυπώνει `Input error!`.

Στην δυαδική κωδικοποίηση, το πρώτο εικονοστοιχείο κάθε ομάδας των οκτώ αποθηκεύεται στο πιο σημαντικό bit του byte. Αν η είσοδος είναι έγχρωμη εικόνα, το πρόγραμμά σας εφαρμόζει πρώτα τον τύπο της φωτεινότητας και μετά το κατώφλι, οπότε η απευθείας μετατροπή πρέπει να δίνει ακριβώς τα ίδια bytes με την σωλήνωση `./photoula --to pgm | ./photoula --to pbm`. Παραδείγματα εκτέλεσης ακολουθούν, ενώ οι εικόνες που προκύπτουν φαίνονται στα Σχήματα 3, 4 και 5.

```
$ ./photoula --to pbm < NIbin.pgm > NIbin.pbm
$ ./photoula --to pgm < MVbin.ppm > MVbin.pgm
$ ./photoula --to pbm < MVbin.pgm > MVbin.pbm
$ ./photoula --to pgm < ISbin.ppm > ISbin.pgm
$ ./photoula --to pbm < ISbin.ppm > ISbin.pbm
$ ./photoula --to pgm < ISbin.ppm | ./photoula --to pbm > ISbin.pbm
$ ./photoula --to pgm < house.ppm > house.pgm
$ ./photoula --to pbm < house.pgm > house.pbm
$ ./photoula --to pgm < colorsasc.ppm > colorsasc.pgm
$ ./photoula --to pbm < colorsasc.pgm > colorsasc.pbm
```

Όλα τα αρχεία που φαίνονται στα παραδείγματα της εκφώνησης της εργασίας μπορείτε να τα κατεβάσετε από το https://github.com/progintro/netpbm [16]. Το τελευταίο παράδειγμα (Σχήμα 6) είναι σε μορφή κειμένου, οπότε μπορείτε να ανοίξετε τα αρχεία του και με έναν επεξεργαστή κειμένου—όπως θα έκανε και ο παραλήπτης ενός email του 1985!



NIbin.ppm



NIbin.pgm



NIbin.pbm

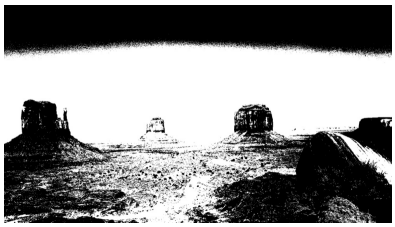
Σχήμα 3: Τοπίο πριν και μετά το φίλτρο. Το πράσινο και το μπλε έχουν σχεδόν εξαφανιστεί μετά την εφαρμογή του κατωφλίου.



MVbin.ppm



MVbin.pgm



MVbin.pbm

Σχήμα 4: Το Monument Valley [6] πριν και μετά τις εφαρμογές του φίλτρου.



ISbin.ppm



ISbin.pgm



ISbin.pbm

Σχήμα 5: Ο καταρράκτης Gullfoss [2] της Ισλανδίας σε τρεις εκδοχές.



colorsasc.ppm



colorsasc.pgm



colorsasc.pbm

Σχήμα 6: Το φίλτρο μας σε μια παλέτα χρωμάτων!

Αλλαγή Κωδικοποίησης και Βάρη Φωτεινότητας (Bonus 30% της βαθμολογίας)

Αν το πρόγραμμά σας δουλεύει ήδη σωστά, μπορείτε να δοκιμάσετε να προσθέσετε ακόμα δύο λειτουργίες (features) για επιπλέον βαθμολογία.

Η πρώτη λειτουργία (20% της βαθμολογίας) είναι το όρισμα `--encoding ascii|binary`, με το οποίο ο χρήστης διαλέγει την κωδικοποίηση της εξόδου, ανεξάρτητα από αυτήν της εισόδου. Τα δύο ορίσματα δουλεύουν ανεξάρτητα: το `--to` ορίζει τον τύπο (format) της εξόδου και το `--encoding` την κωδικοποίησή της, οπότε κάθε μετατροπή του πίνακα των μετατροπών επιτρέπεται με οποιαδήποτε κωδικοποίηση. Για παράδειγμα, με `--to ppm --encoding ascii` μια έγχρωμη εικόνα σε δυαδική κωδικοποίηση (P6) γίνεται έγχρωμη εικόνα σε μορφή κειμένου (P3), ενώ με `--to pbm --encoding ascii` μια εικόνα με αποχρώσεις του γκρι σε δυαδική κωδικοποίηση (P5) γίνεται ασπρόμαυρη εικόνα σε μορφή κειμένου (P1). Αν η κωδικοποίηση που ζητείται είναι ίδια με αυτήν της εισόδου, το όρισμα δεν αλλάζει την συμπεριφορά του προγράμματος.

Η δεύτερη λειτουργία (10% της βαθμολογίας) είναι να αλλάζουμε τον ίδιο τον τύπο της φωτεινότητας. Τα ορίσματα `--r R`, `--g G` και `--b B` ορίζουν τους συντελεστές r, g και b του τύπου, ο οποίος γενικεύεται σε:

$$\left\lfloor \frac{r \cdot R + g \cdot G + b \cdot B}{r + g + b} \right\rfloor$$

Το κάθε όρισμα είναι προαιρετικό και οι προεπιλεγμένες τιμές του είναι $r = 299$, $g = 587$ και $b = 114$, οπότε χωρίς ορίσματα παίρνουμε ακριβώς τον αρχικό τύπο. Οι τιμές πρέπει να είναι ακέραιοι στο εύρος $[0, 1000]$ και όχι όλες μηδενικές—διαφορετικά έχουμε λάθος στα ορίσματα. Τα ορίσματα αυτά επιτρέπονται μόνο μαζί με `--to pgm` ή `--to pbm`, ενώ αν η είσοδος δεν είναι έγχρωμη εικόνα το πρόγραμμά σας τυπώνει `Unsupported conversion` και τερματίζει με κωδικό εξόδου 1. Παραδείγματα εκτέλεσης ακολουθούν:

```
$ ./photoula --to ppm --encoding ascii < NIbin.ppm > NIasc.ppm
$ ./photoula --to ppm --encoding binary < NIasc.ppm > NIbin2.ppm
$ cmp NIbin2.ppm <(./photoula --to ppm < NIbin.ppm)
$ ./photoula --encoding ascii --to pbm --threshold 100 \
    < MVbin.pgm > MVasc.pbm
$ ./photoula --to pgm --r 1000 --g 0 --b 0 < MVbin.ppm > MVred.pgm
$ ./photoula --to pgm --r 1 --g 1 --b 1 < MVbin.ppm > MVavg.pgm
```

Δοκιμάστε το: στο `MVred.pgm` κρατάμε μόνο το κόκκινο κανάλι, οπότε οι κόκκινοι βράχοι του Σχήματος 4 φωτίζονται ενώ ο μπλε ουρανός σκοτεινιάζει.

README. Τέλος, πριν την υποβολή της εργασίας, μην ξεχάσετε το photoula/README.md, μέσα στο οποίο πρέπει να συμπεριλάβετε τις όποιες παρατηρήσεις σας κατά την διεκπεραίωση της άσκησης. Ο κώδικας απαιτείται να είναι καλά τεκμηριωμένος με σχόλια καθώς αυτό θα είναι μέρος της βαθμολόγησης.

Καλή Επιτυχία!

Αναφορές

[1] Grayscale. https://en.wikipedia.org/wiki/Grayscale.

[2] Gullfoss. https://en.wikipedia.org/wiki/Gullfoss.

[3] IrfanView. https://www.irfanview.com/.

[4] Jef Poskanzer. https://en.wikipedia.org/wiki/Jef_Poskanzer.

[5] Luma (video). https://en.wikipedia.org/wiki/Luma_(video).

[6] Monument Valley. https://en.wikipedia.org/wiki/Monument_Valley.

[7] Netpbm. https://en.wikipedia.org/wiki/Netpbm.

[8] Netpbm History. https://netpbm.sourceforge.net/history.html.

[9] PBM Format Specification. https://netpbm.sourceforge.net/doc/pbm.html.

[10] PGM Format Specification. https://netpbm.sourceforge.net/doc/pgm.html.

[11] Pixel. https://en.wikipedia.org/wiki/Pixel.

[12] PPM Format Specification. https://netpbm.sourceforge.net/doc/ppm.html.

[13] Rec. 601. https://en.wikipedia.org/wiki/Rec._601.

[14] RGB Color Model. https://en.wikipedia.org/wiki/RGB_color_model.

[15] Thresholding (image processing). https://en.wikipedia.org/wiki/Thresholding_(image_processing).

[16] DIT. Αρχεία εικόνων της εργασίας . https://github.com/progintro/netpbm.

[17] DIT. Οργανισμός για το μάθημα (GitHub progintro) . https://github.com/progintro.

[18] DIT. Πρόσκληση για Εργασία 1 . https://hw.uoa.gr/courses/progintro/assignments/hw1-2026/accept.